Guides

Function calling

Connect the xAI models to external tools and systems to build AI assistants and various integrations.

With stream response, the function call will be returned in whole in a single chunk, instead of being streamed across chunks.

Introduction

Function calling enables language models to use external tools, which can intimately connect models to digital and physical worlds.

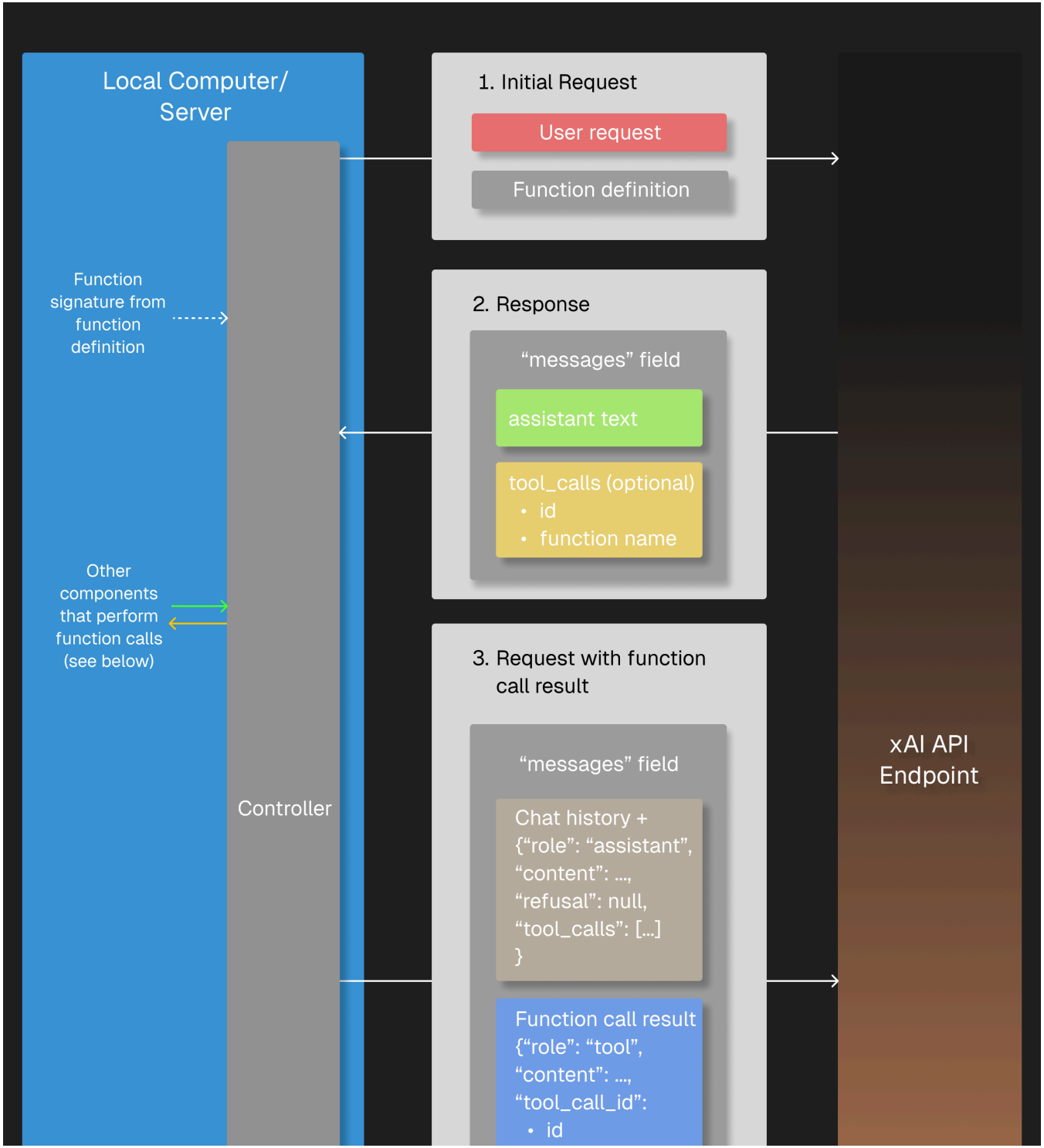
This is a powerful capability that can be used to enable a wide range of use cases.

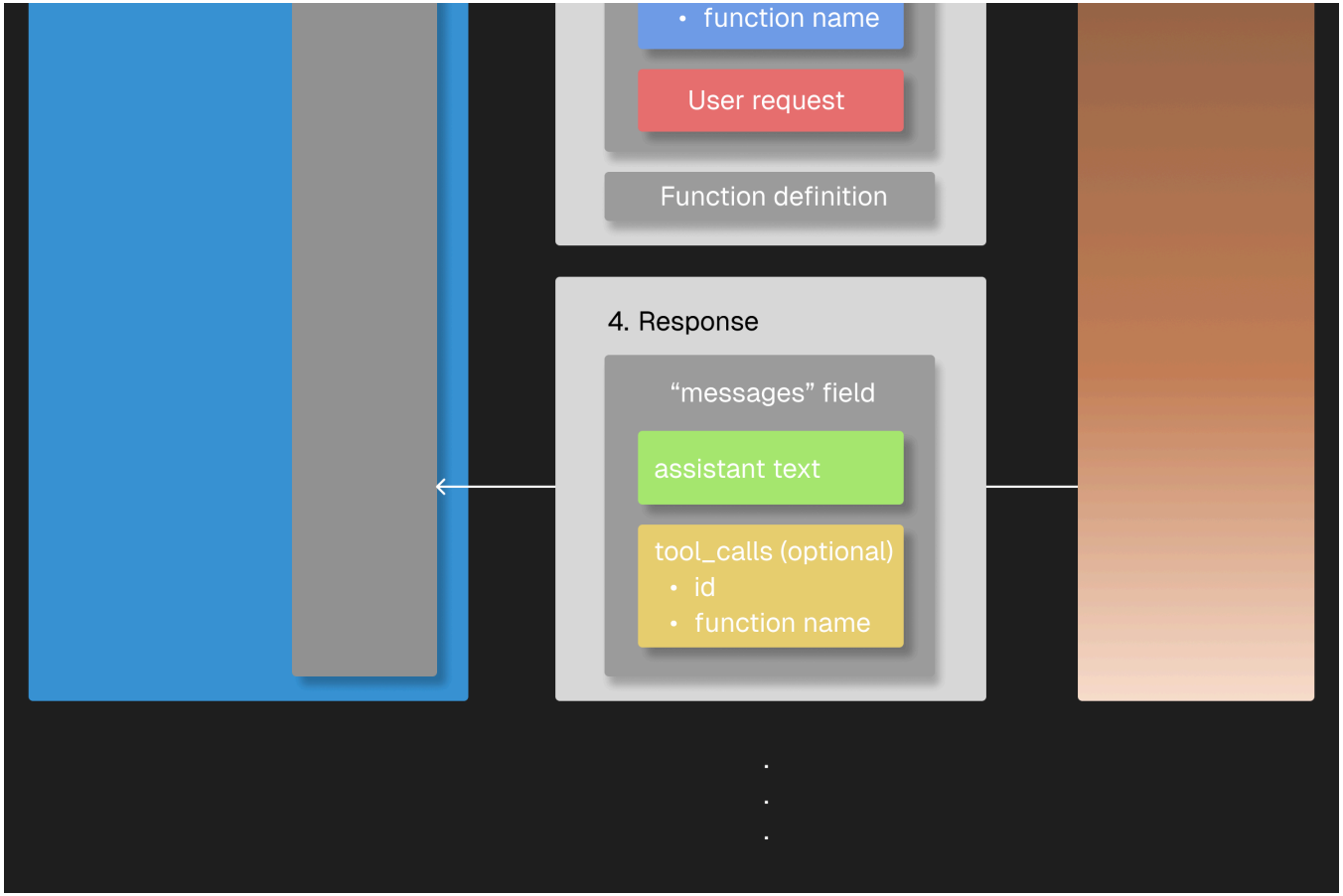
- Calling public APIs for actions ranging from looking up football game results to getting real-time satellite positioning data
- Analyzing internal databases
- Browsing web pages
- Executing code

- Interacting with the physical world (e.g. booking a flight ticket, opening your tesla car door, controlling robot arms)

Walkthrough

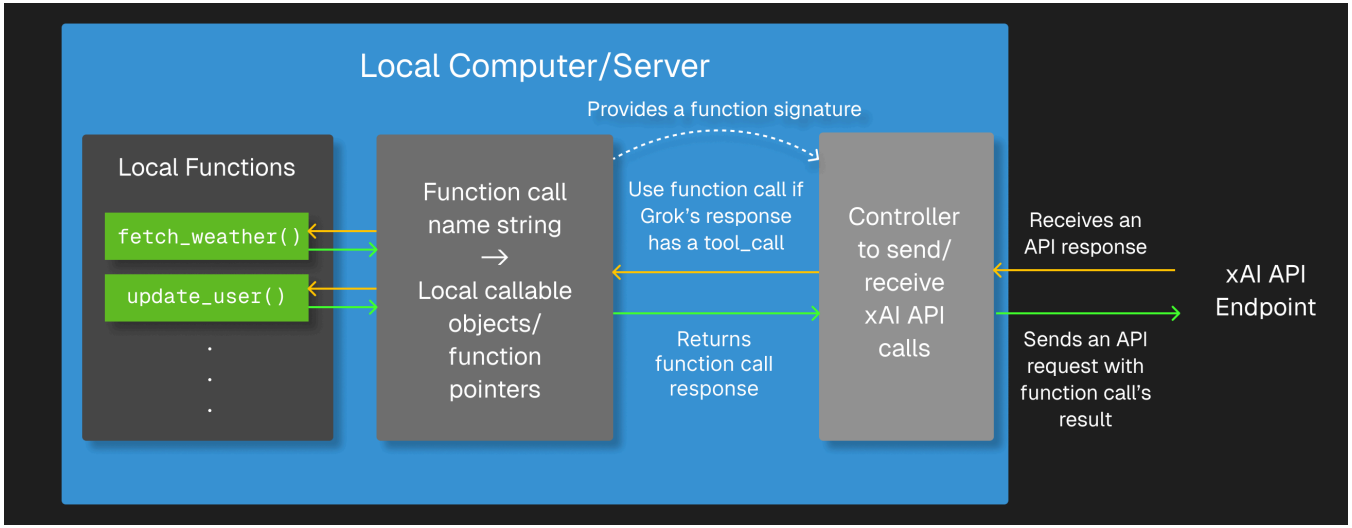
The request/response flow for function calling can be demonstrated in the following illustration.


Local Computer/ Server
Function signature from function definition
Other components that perform function calls (see below)
Controller
1. Initial Request
User request
Function definition
2. Response
"messages" field
assistant text
tool_calls (optional)
• id
• function name
3. Request with function call result
"messages" field
Chat history +
{"role": "assistant",
"content": ...,
"refusal": null,
"tool_calls": [...]
}
Function call result
{"role": "tool",
"content": ...,
"tool_call_id":
• id
xAI API Endpoint



You can think of it as the LLM initiating RPCs (Remote Procedure Calls) to user system. From the LLM's perspective, the "2. Response" is an RPC request from LLM to user system, and the "3. Request" is an RPC response with information that LLM needs.

One simple example of a local computer/server, where the computer/server determines if the response from Grok contains a `tool_call`, and calls the locally-defined functions to perform user-defined actions:



The whole process looks like this in pseudocode:

```pseudocode
// ... Define tool calls and their names

messages = []

/* Step 1: Send a new user request */

messages += {<new user request message>}
response = send_request_to_grok(message)

messages += response.choices[0].message  // Append assistant response

while (true) {
    /* Step 2: Run tool call and add tool call result to messages */
    if (response contains tool_call) {
        // Grok asks for tool call

        for (tool in tool_calls) {
            tool_call_result = tool(arguments provided in response) // Perform tool call
            messages += tool_call_result  // Add result to message
        }
    }
```

```
    read(user_request)

    if (user_request) {
        messages += {<new user request message>}
    }

    /* Step 3: Send request with tool call result to Grok*/
    response = send_request_to_grok(message)

    print(response)
}
```

We will demonstrate the function calling in the following Python script. First, let's create an API client:

```python
python (xAI SDK)

import os
import json

from xai_sdk import Client
from xai_sdk.chat import tool, tool_result, user

client = Client(api_key=os.getenv('XAI_API_KEY'))
chat = client.chat.create(model="grok-4")
```

Preparation - Define tool functions and function mapping

Define tool functions as callback functions to be called when model requests them in response.

Normally, these functions would either retrieve data from a database, or call another API endpoint, or perform some actions. For demonstration purposes, we hardcode to return 59° Fahrenheit/15° Celsius as the temperature, and 15,000 feet as the cloud ceiling.

The parameters definition will be sent in the initial request to Grok, so Grok knows what tools and parameters are available to be called.

To reduce human error, you can define the tools partially using Pydantic.

Function definition using Pydantic:

```python (xAI SDK)
from typing import Literal

from pydantic import BaseModel, Field


class TemperatureRequest(BaseModel):
    location: str = Field(description="The city and state, e.g. San Francisco, CA")
    unit: Literal["celsius", "fahrenheit"] = Field(
        "fahrenheit", description="Temperature unit"
    )

class CeilingRequest(BaseModel):
    location: str = Field(description="The city and state, e.g. San Francisco, CA")

def get_current_temperature(request: TemperatureRequest):
    temperature = 59 if request.unit.lower() == "fahrenheit" else 15
    return {
        "location": request.location,
        "temperature": temperature,
        "unit": request.unit,
    }

def get_current_ceiling(request: CeilingRequest):
    return {
        "location": request.location,
        "ceiling": 15000,
        "ceiling_type": "broken",
        "unit": "ft",
    }


# Generate the JSON schema from the Pydantic models
```

```python
get_current_temperature_schema = TemperatureRequest.model_json_schema()
get_current_ceiling_schema = CeilingRequest.model_json_schema()

# Definition of parameters with Pydantic JSON schema
tool_definitions = [
    tool(
        name="get_current_temperature",
        description="Get the current temperature in a given location",
        parameters=get_current_temperature_schema,
    ),
    tool(
        name="get_current_ceiling",
        description="Get the current cloud ceiling in a given location",
        parameters=get_current_ceiling_schema,
    ),
]
```

Function definition using raw dictionary:

python (xAI SDK)

```python
from typing import Literal


def get_current_temperature(location: str, unit: Literal["celsius", "fahrenheit"] = "fahrenheit"):
    temperature = 59 if unit == "fahrenheit" else 15
    return {
        "location": location,
        "temperature": temperature,
        "unit": unit,
    }


def get_current_ceiling(location: str):
    return {
        "location": location,
        "ceiling": 15000,
        "ceiling_type": "broken",
        "unit": "ft",
    }
```

```python
# Raw dictionary definition of parameters
tool_definitions = [
    tool(
        name="get_current_temperature",
        description="Get the current temperature in a given location",
        parameters={
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "The city and state, e.g. San Francisco, CA",
                },
                "unit": {
                    "type": "string",
                    "enum": ["celsius", "fahrenheit"],
                    "default": "fahrenheit",
                },
            },
            "required": ["location"],
        },
    ),
    tool(
        name="get_current_ceiling",
        description="Get the current cloud ceiling in a given location",
        parameters={
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "The city and state, e.g. San Francisco, CA",
                }
            },
            "required": ["location"],
        },
    ),
]
```

Create a string → function mapping, so we can call the function when model sends it's name. e.g.

```python

tools_map = {
    "get_current_temperature": get_current_temperature,
```

```
    "get_current_ceiling": get_current_ceiling,
}
```

---

1. Send initial message

With all the functions defined, it's time to send our API request to Grok!

Now before we send it over, let's look at how the generic request body for a new task looks like.

Here we assume a previous tool call has Note how the tool call is referenced three times:

- By `id` and `name` in "Mesage History" assistant's first response
- By `tool_call_id` in "Message History" tool's content
- In the `tools` field of the request body

New Request Request Body

"messages":

Message History

```
[{
    "role": "user",
    "content": "What's the
temperature in Paris?"
},
{
    "role": "assistant",
    "content": ...,
    "refusal": null,
    "tool_calls": [
        {"id": "call_00001",
         "function":
             {"arguments":
"{'location': 'Paris'}",
"name": "get_current_temperature"},
"type": "function"}]
},
{
    "role": "tool",
    "content": '{"location":
"Paris", "temperature": 15, "unit":
"fahrenheit"}'
    "tool_call_id": "call_00001"
},
{
    "role": "assistant",
    "content": "The current
temperature in Paris, France is
15°F."
},
```

New User Request

```
{
    "role": "user",
```

Now we compose the request messages in the request body and send it over to Grok. Grok should return a response that asks us for a tool call.

```
python (xAI SDK)

chat = client.chat.create(
    model="grok-4",
    tools=tool_definitions,
    tool_choice="auto",
)
chat.append(user("What's the temperature like in San Francisco?"))
response = chat.sample()

# You can inspect the response tool calls which contains a tool call
print(response.tool_calls)
```

---

2. Run tool functions if Grok asks tool call and append function returns to message

We retrieve the tool function names and arguments that Grok wants to call, run the functions, and add the result to messages.

At this point, you can choose to **only respond to tool call with results** or **add a new user message request.**

The `tool` message would contain the following:

```
json

{
    "role": "tool",
    "content": <json string of tool function's returned object>,
    "tool_call_id": <tool_call.id included in the tool call response by Grok>,
}
```

The request body that we try to assemble and send back to Grok. Note it looks slightly different from the new task request body:

Request Body in Response to Function Call

```
"messages":
```

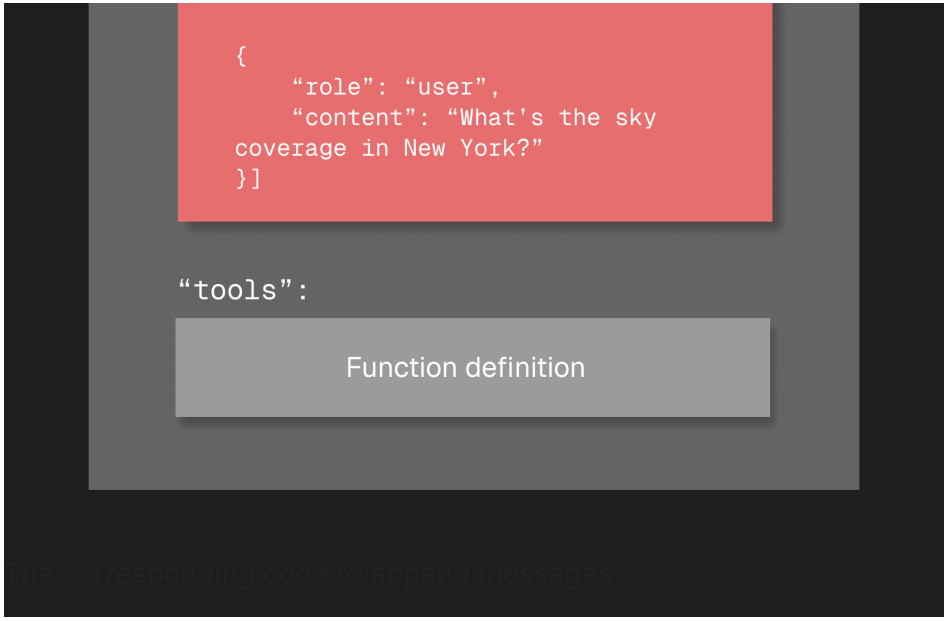
Message History

```
[
...
{
    "role": "user",
    "content": "What's the
temperature in San Francisco?"
},
{
    "role": "assistant",
    "content": ...,
    "refusal": null,
    "tool_calls": [
        {"id": "call_00002",
         "function":
             {"arguments":
"{'location': 'San Francisco'}",
"name": "get_current_temperature"},
"type": "function"}]
}
```

Function Call Result

```
{
    "role": "tool",
    "content": '{"location": "San
Francisco", "temperature": 15,
"unit": "fahrenheit"}'
    "tool_call_id": "call_00002"
}
```

(Optional) New User Request in Parallel while Responding to Function Call



The corresponding code to append messages:

python (xAI SDK)

```python
# Append assistant message including tool calls to messages
chat.append(response)

# Check if there is any tool calls in response body
# You can also wrap this in a function to make the code cleaner

if response.tool_calls:
    for tool_call in response.tool_calls:

        # Get the tool function name and arguments Grok wants to call
        function_name = tool_call.function.name
        function_args = json.loads(tool_call.function.arguments)

        # Call one of the tool function defined earlier with arguments
        result = tools_map[function_name](**function_args)

        # Append the result from tool function call to the chat message history
        chat.append(tool_result(result))
```

3. Send the tool function returns back to the model to get the response

```python (xAI SDK)
response = chat.sample()
print(response.content)
```

---

4. (Optional) Continue the conversation

You can continue the conversation following Step 2. Otherwise you can terminate.

Function calling modes

By default, the model will automatically decide whether a function call is necessary and select which functions to call, as determined by the `tool_choice: "auto"` setting.

We offer three ways to customize the default behavior:

1. To force the model to always call one or more functions, you can set `tool_choice: "required"`. The model will then always call function. Note this could force the model to hallucinate parameters.
2. To force the model to call a specific function, you can set `tool_choice: {"type": "function", "function": {"name": "my_function"}}`.
3. To disable function calling and force the model to only generate a user-facing message, you can either provide no tools, or set `tool_choice: "none"`.

Parallel function calling

By default, parallel function calling is enabled, so you can process multiple function calls in one request/response cycle. When two or more tool calls are required, all of the tool call requests will be included in the response body. You can disable it by setting `parallel_function_calling : "false"` .